SENIOR SOLUTION ARCHITECT · PROJECT GUIDE

Tienda de *Radiadores* Sistema Integrado

Guía técnica de arquitectura, distribución de carga de trabajo y contratos de integración para un equipo de 4 personas bajo stack .NET.

PERSONA 1
🖥️ Caja (POS)
Frontend Desktop / WinForms / WPF

PERSONA 2
🗄️ Core
Backend API + SQL Server Central

PERSONA 3
⚡ Integración
Middleware · NServiceBus · Sync

PERSONA 4
🌐 Website
Portal Web Cliente · ASP.NET / Blazor

ÍNDICE DE CONTENIDOS

01 Arquitectura de Comunicación
02 Backlog por Persona
03 Estrategia Offline / Sincronización
04 Recomendaciones de Stack
05 Contratos de API & Mensajes
06 Seguridad & Auditoría

SECCIÓN 01

Arquitectura de Comunicación

El sistema opera en **cuatro capas desacopladas**. La regla cardinal es que **ningún canal (Caja, Website) accede directamente al Core**; toda comunicación pasa por la capa de Integración, que actúa como proxy inteligente con capacidad de respuesta autónoma en caso de caída del Core.

WPF / WinForms
LocalDB / SQLite
NServiceBus Local

ASP.NET Core
NServiceBus Host
SQL Express Mirror

ASP.NET Core
SQL Server Central
EF Core + Identity

P4 · WEBSITE

Web Portal

Blazor / Razor Pages
ASP.NET Core MVC

REST → Integración

El Website consume los mismos endpoints expuestos por Integración que consume la Caja. Un solo contrato de API sirve ambos canales.

Flujo Online Nominal

Caja → POST /api/ventas a Integración. Integración valida, enruta al Core, recibe respuesta, registra en su log local y retorna resultado a la Caja. La Caja imprime recibo local. El Core guarda la transacción definitiva y emite evento de confirmación vía NServiceBus.

Flujo Offline (degradado)

Si Integración detecta que el Core no responde (circuit breaker abierto), atiende la solicitud desde su **espejo local de inventario**, persiste la transacción en su propia base de datos y encola el mensaje NServiceBus en la cola local. Al restablecer la conexión, el Saga de NServiceBus drena la cola y aplica las transacciones en el Core manteniendo idempotencia.

⚠ **Restricción de diseño:** El Core es la única fuente de verdad. Todo inventario en Integración y en la Caja es un caché con TTL. Las transacciones offline son eventuales, nunca definitivas hasta confirmación del Core.

SECCIÓN 02

Backlog de Responsabilidades

Persona 1 — Caja POS

WPF · WinForms · LocalDB · ReportViewer local

→ CRÍTICO Login cajero con hash bcrypt; sesión bloqueada por inactividad

Persona 2 — Core Backend

ASP.NET Core · SQL Server · EF Core · Identity

→ CRÍTICO ASP.NET Identity + JWT: roles (Admin, Mantenimiento, Consulta), política de contraseñas

- → Asignación de cajero a sucursal; validar contra Integración al iniciar
- → CRÍTICO Inicio de día: registrar monto inicial, sincronizar inventario local
- → Transacciones de entrada y salida de efectivo con motivo y firma digital
- → CRÍTICO Pantalla de venta: búsqueda de producto, carrito, descuentos, pago
- → Impresión de recibo local (ReportViewer / RDLC local) sin depender del Core
- → CRÍTICO Modo offline: persistir ventas en SQLite/LocalDB con flag "pendiente de sync"
- → Monitor de cola offline: mostrar transacciones pendientes y su estado
- → Cierre de día: cuadre automático de efectivo vs. sistema
- → Registro de auditoría local (Serilog → archivo .log rotativo)
- → Consumir servicios de Integración via HttpClient con Polly (retry + CB)
- → Pantalla de configuración: URL de Integración, certificados, sucursal

`WPF / WinForms` `LocalDB / SQLite` `HttpClient + Polly` `ReportViewer RDLC` `NServiceBus.LearningTransport` `Serilog (File)` `BCrypt.Net`

- → CRUD Usuarios, Roles y Perfiles con auditoría de cada operación
- → Crear el "Cliente Anónimo" como registro predeterminado en BD (ID fijo)
- → CRÍTICO CRUD Productos (código, descripción, precio, stock, categoría)
- → CRUD Clientes (RNC/cédula, nombre, contacto, límite crédito)
- → CRUD Servicios (mano de obra, instalación, diagnóstico)
- → CRUD Cotizaciones → flujo a Factura
- → CRÍTICO CRUD Facturas: numeración secuencial, itbis, líneas de detalle
- → Cuentas por Cobrar: generación automática desde factura, estados, abonos
- → CRÍTICO API de servicios REST (versionada v1) para consumo de Integración
- → Middleware de auditoría: guarda Request/Response completo en tabla AuditLog
- → Generación de reportes finales con Microsoft ReportViewer (server-side SSRS/RDLC)
- → Migrations EF Core, seed de datos iniciales, índices de performance
- → Health checks: /health endpoint para circuit breaker de Integración

`ASP.NET Core 8` `EF Core + Migrations` `SQL Server Central` `ASP.NET Identity` `JWT Bearer` `ReportViewer SSRS` `Serilog (SQL + File)` `AutoMapper` `FluentValidation`

⚡ Persona 3 — Integración

🌐 Persona 4 — Website Cliente

Blazor / ASP.NET MVC · Razor Pages

ASP.NET Core · NServiceBus · SQL Express Mirror

- → CRÍTICO API Gateway: exponer todos los endpoints hacia Caja y Website
- → CRÍTICO Circuit Breaker (Polly): detectar caída del Core y activar modo degradado
- → Espejo de inventario local en SQL Express: sincronización periódica (background service)
- → CRÍTICO NServiceBus Saga: gestionar transacciones offline, encolado y replay
- → Publicar endpoints de autenticación (proxy JWT hacia Core)
- → Publicar servicios de inventario, ventas, pagos, cotizaciones
- → Registro completo de peticiones y respuestas (tabla IntegrationLog + Serilog)
- → Proceso de reconciliación: al restablecer Core, aplicar transacciones offline con idempotencia
- → Consumir APIs externas: procesadores de pago, suplidores (adaptadores)
- → Refrescar inventario mirror cada N minutos (configurable vía appsettings)
- → Manejo de errores con dead-letter queue para mensajes que fallan repetidamente
- → Servicio de notificaciones: enviar estados a Caja y Website (SignalR o polling)

`ASP.NET Core 8` `NServiceBus` `SQL Server Express` `Polly (CB + Retry)` `Hangfire / BGService` `Serilog (SQL)` `Ocelot / YARP` `SignalR`

- → Login de cliente con usuario/clave (consume /auth de Integración)
- → Catálogo público de productos y servicios (sin login requerido para ver)
- → Filtros de búsqueda: categoría, precio, disponibilidad
- → CRÍTICO Carrito de compras y flujo de orden completa
- → Simulación de pago: integración con procesador externo vía Integración
- → Estado de la orden: pantalla de seguimiento con estados (Pendiente, Procesando, Listo)
- → Historial de órdenes del cliente autenticado
- → Diseño responsivo (Bootstrap / Tailwind) mobile-first
- → Manejo de sesión: tokens JWT almacenados en HttpOnly cookies
- → Logging de acciones de usuario (Serilog → Application Insights opcional)
- → Página de error amigable sin exponer detalles técnicos

`Blazor Server / WASM` `Razor Pages` `HttpClient (Typed)` `JWT (HttpOnly Cookie)` `Bootstrap 5` `Serilog` `FluentValidation`

SECCIÓN 03

Estrategia Offline & Sincronización

El requerimiento exige que la Caja opere sin conexión al Core y que Integración pueda responder autónomamente. La estrategia combina **espejo de datos local**, **NServiceBus para mensajería durable** y **Sagas para orquestación de compensación**.

🔌 Fase 1 — Detección de Pérdida de Conexión

1 Health Check Continuo (Integración → Core)

Un **BackgroundService** en Integración hace ping a `GET /health` del Core cada 15 segundos. Polly gestiona un **Circuit Breaker**: se abre tras 3 fallos consecutivos, permanece abierto 60s, luego entra en Half-Open.

```
// Registro en Program.cs de Integración builder.Services.AddHttpClient("CoreApi", c =>
c.BaseAddress = new Uri(coreUrl)) .AddTransientHttpErrorPolicy(p =>
p.CircuitBreakerAsync(3, TimeSpan.FromSeconds(60)));
```

2 Activación del Modo Degradado

Cuando el Circuit Breaker está abierto, Integración levanta la flag `CoreAvailable = false` (almacenada en IMemoryCache / Redis). Todos los endpoints expuestos hacia la Caja usan el espejo local en lugar del Core.

3 Espejo de Inventario (SQL Express)

Un Job (Hangfire) sincroniza productos, precios y stock desde el Core cada **5 minutos** cuando hay conexión. En modo offline, Integración sirve desde este espejo. Los descuentos de stock se aplican optimistamente y se reconcilian después.

```
// Tabla mirror en Integración CREATE TABLE ProductoMirror ( Id INT PRIMARY KEY, Codigo
NVARCHAR(50), Nombre NVARCHAR(200), Precio DECIMAL(18,2), Stock INT, UltimaSync
DATETIME2, EsActivo BIT );
```

📦 Fase 2 — Encolado de Transacciones Offline (NServiceBus)

4 Caja → Cola Local NServiceBus

Cuando Integración no está disponible, la Caja persiste la venta en su **LocalDB** con estado `Pendiente` y publica un mensaje `VentaRealizadaOfflineMessage` en la cola local de NServiceBus (Learning Transport para desarrollo, MSMQ/RabbitMQ para producción).

```
// Mensaje de dominio public record VentaRealizadaOfflineMessage : ICommand { public Guid IdTransaccionLocal { get; init; } public string IdCajero { get; init; } public string IdSucursal { get; init; } public List<LineaVentaDto> Lineas { get; init; } public decimal MontoTotal { get; init; } public string MetodoPago { get; init; } public DateTimeOffset FechaLocal { get; init; } }
```

5

Integración → Saga de Sincronización

Al restablecerse la comunicación (Circuit Breaker pasa a Closed), el EndpointHost de NServiceBus en Integración procesa los mensajes acumulados. Un **Saga** orquesta el reenvío idempotente al Core, reintentando con backoff exponencial.

```
// Saga simplificada public class SyncOfflineSaga : Saga<SyncOfflineSagaData>, IAmStartedByMessages<VentaRealizadaOfflineMessage>, IHandleMessages<VentaAplicadaEnCoreEvent>, IHandleTimeouts<RetryTimeout> { public async Task Handle(VentaRealizadaOfflineMessage msg, IMessageHandlerContext ctx) { // 1. Verificar que Core está disponible // 2. Enviar POST /api/v1/ventas al Core con IdempotencyKey = msg.IdTransaccionLocal // 3. Si falla → RequestTimeout<RetryTimeout> (ctx, TimeSpan.FromMinutes(5)) // 4. Si éxito → marcar saga como completada, publicar VentaSincronizadaEvent } }
```

6

Core — Idempotencia de Transacciones

El Core debe aceptar una cabecera `Idempotency-Key` en cada POST de venta. Si ya existe una factura con ese GUID, devuelve **200 OK** con el resultado original sin crear un duplicado. Esto se implementa con una tabla `IdempotencyLog`.

🔄 Fase 3 — Reconciliación y Notificación

Una vez aplicadas todas las transacciones offline, Integración publica un evento `ReconciliacionCompletadaEvent` que la Caja escucha. La Caja actualiza el estado de sus ventas locales de `Pendiente` → `Sincronizada` y muestra una notificación al cajero.

✓ **Resultado:** El cajero nunca pierde una transacción. El Core siempre termina con la verdad. El inventario en el espejo de Integración se refresca inmediatamente tras la reconciliación.

SECCIÓN 04

Recomendaciones de Stack Adicional

FluentValidation

Centraliza las reglas de validación de DTOs y Commands fuera de los controladores. Permite validar complejidad de contraseñas, formato de RNC/cédula, rangos de precios y cualquier regla de negocio de forma declarativa y testeable. Se integra nativamente con ASP.NET Core y el pipeline de ModelState.

Usar en: **P2 (Core), P3 (Integración), P4 (Website)**

AutoMapper

Elimina el mapeo manual tedioso entre Entities, DTOs y ViewModels. Reduce bugs por omisión de campos y acelera el desarrollo en las capas de Core e Integración. Se configura con Profiles que documentan implícitamente la estructura del contrato de datos entre capas.

Usar en: **P2 (Core), P3 (Integración)**

Microsoft.Extensions.Diagnostics.HealthChecks + Seq

HealthChecks expone un endpoint estándar que Polly en Integración puede consultar para gestionar el Circuit Breaker de forma precisa (BD, disco, dependencias externas). **Seq** es un servidor centralizado de logs estructurados Serilog que permite buscar, filtrar y alertar sobre eventos de auditoría en producción sin costo adicional en desarrollo.

Usar en: **P1, P2, P3, P4 (todas las capas)**

Adicional recomendado: Polly v8 (ya mencionado en la arquitectura) para retry, circuit breaker, timeout y bulkhead en todas las llamadas HTTP entre capas. Es la librería estándar de resiliencia en el ecosistema .NET y ya está integrada con `IHttpClientFactory`.

SECCIÓN 05

Contratos de API & Mensajes

Estos contratos deben acordarse **antes** de comenzar el desarrollo paralelo. El equipo puede usar un archivo `contracts.yaml` en un repositorio compartido (o una colección Postman/OpenAPI) como fuente de verdad. La Persona 3 (Integración) es la propietaria de estos contratos.

Servicios REST expuestos por Integración → consumidos por Caja y Website

MÉTODO	ENDPOINT	DESCRIPCIÓN	PROPIETARIO	CONSUMIDOR
POST	`/api/v1/auth/login`	Autenticación, devuelve JWT + refresh token. Valida rol del usuario.	P3 (proxy P2)	P1, P4
POST	`/api/v1/auth/refresh`	Renovación de token expirado sin re-login.	P3	P1, P4
GET	`/api/v1/productos`	Lista paginada con filtros (nombre, código, categoría). Sirve desde mirror en modo offline.	P3	P1, P4
GET	`/api/v1/productos/{id}`	Detalle de producto con stock disponible.	P3	P1, P4
GET	`/api/v1/servicios`	Lista de servicios ofrecidos (mano de obra, diagnóstico, instalación).	P3	P1, P4
GET	`/api/v1/clientes/{id}`	Datos del cliente. Si no existe, retorna el cliente anónimo predeterminado.	P3	P1
POST	`/api/v1/ventas`	Crear venta/factura. Body incluye líneas, cliente, método de pago. Header: `Idempotency-Key`.	P3 → P2	P1
POST	`/api/v1/caja/inicio-dia`	Registrar apertura de caja con monto inicial. Retorna ID de sesión de caja.	P3	P1
POST	`/api/v1/caja/movimiento`	Entrada o salida de efectivo. Body: {tipo: IN|OUT, monto, motivo}.	P3	P1

MÉTODO	ENDPOINT	DESCRIPCIÓN	PROPIETARIO	CONSUMIDOR
POST	`/api/v1/caja/cierre-dia`	Cuadre de caja: retorna diferencias entre efectivo contado y transacciones.	P3	P1
POST	`/api/v1/ordenes`	Crear orden desde el portal web. Flujo asíncrono, retorna ID de orden.	P3 → P2	P4
GET	`/api/v1/ordenes/{id}/estado`	Consultar estado de la orden del cliente (Pendiente, Procesando, Listo, Entregado).	P3	P4
POST	`/api/v1/pagos/simular`	Simular pago de una orden (para el portal web). Integra con procesador externo.	P3	P4
GET	`/api/v1/cuentas-cobrar/{clienteId}`	CxC pendientes del cliente. Permite hacer abonos vía POST.	P3 → P2	P1
GET	`/health`	Estado de Integración: Core disponible, mirror actualizado, cola pendiente.	P3	P1, P4, Monitor

Mensajes NServiceBus (Bus de Eventos)

TIPO	MENSAJE	PUBLICADO POR	CONSUMIDO POR	PROPÓSITO
CMD	`VentaRealizadaOfflineMessage`	P1 (Caja)	P3 (Integración)	Encolar venta cuando Integración no está disponible.
EVT	`VentaSincronizadaEvent`	P3	P1	Confirmar que la venta offline fue aplicada en el Core.
EVT	`InventarioActualizadoEvent`	P2 (Core)	P3	Notificar cambios de stock para refrescar el mirror.

TIPO	MENSAJE	PUBLICADO POR	CONSUMIDO POR	PROPÓSITO
CMD	`AplicarTransaccionesOfflineCommand`	P3 (Saga)	P2 (Core API)	Reenviar lote de transacciones offline al Core tras reconexión.
EVT	`OrdenCambioEstadoEvent`	P2	P3 → P4	Propagar cambios de estado de orden al portal web.
EVT	`ReconciliacionCompletadaEvent`	P3	P1	Notificar a Caja que todas las transacciones offline están sincronizadas.

⚠ **Regla de contratos:** Ningún campo de estos contratos debe cambiar sin coordinación entre las partes. Se recomienda versionar los endpoints (`/api/v1/`) y usar una OpenAPI spec (`swagger.json`) como contrato formal. La Persona 3 es el árbitro de breaking changes.

SECCIÓN 06

Seguridad & Auditoría

Cada capa debe implementar estas capas de seguridad de forma independiente. La seguridad en profundidad (*defense in depth*) asegura que un fallo en una capa no comprometa el sistema completo.

🔑 Autenticación & Autorización

ASP.NET Identity con JWT. Políticas de contraseña: mínimo 10 caracteres, mayúsculas, números y símbolo. Roles: Admin, Mantenimiento, Consulta, Cajero. Tokens con expiración de 1 hora + refresh. Caja usa certificado de cliente adicional.

📝 Auditoría de Acciones (Core)

Middleware global que intercepta cada request con usuario autenticado, acción, timestamp, IP origen, payload (sanitizado) y respuesta. Guardado en tabla `AuditLog` inmutable (INSERT-only, sin UPDATE/DELETE permitidos al rol de la app).

🔌 Auditoría de Integración

Integración registra en tabla `IntegrationLog`: timestamp, endpoint, dirección (IN/OUT), cuerpo de petición y respuesta, código HTTP, latencia y si fue

🖥️ Seguridad en Caja

Bloqueo de pantalla por inactividad (5 min). PIN de confirmación para transacciones de salida de efectivo. Logs rotados con Serilog en archivo local cifrado. La Caja

atendida desde mirror. Nunca loguear datos sensibles como contraseñas o números de tarjeta completos.

nunca almacena contraseñas en texto claro; usa tokens con TTL corto.

🌐 Seguridad en Website

HTTPS obligatorio. CSP headers. Cookies HttpOnly + Secure + SameSite=Strict. Rate limiting en endpoints de login (10 intentos/min por IP). Protección CSRF en formularios. Validación de datos en cliente Y servidor.

🕵️ Cliente Anónimo

El Core define un registro fijo de cliente con ID determinístico (ej. GUID constante o ID = 0) llamado "Cliente Anónimo". La Caja lo selecciona por defecto cuando no se identifica al cliente. Permite facturar sin crear registros de cliente espurios.

Política de Reportes

El **Core es el único generador de reportes oficiales** mediante Microsoft ReportViewer/RDLC con datos del SQL Server central. Estos reportes son los documentos fiscales y gerenciales definitivos.

La **Caja imprime recibos locales** mediante plantillas RDLC ligeras embebidas en la aplicación desktop. Estos recibos son comprobantes operacionales, no documentos fiscales. Deben incluir un número de referencia local que permite correlacionarse con la factura oficial generada por el Core una vez sincronizado.

📌 **Serilog Sinks recomendados por capa —** Core: `Serilog.Sinks.MSSqlServer` + `Serilog.Sinks.Seq`. Integración: `MSSqlServer` + `Seq`. Caja: `Serilog.Sinks.File` (rolling daily). Website: `File` + `Seq` opcional. Todos los eventos deben incluir los campos: `UserId`, `Layer`, `CorrelationId`.